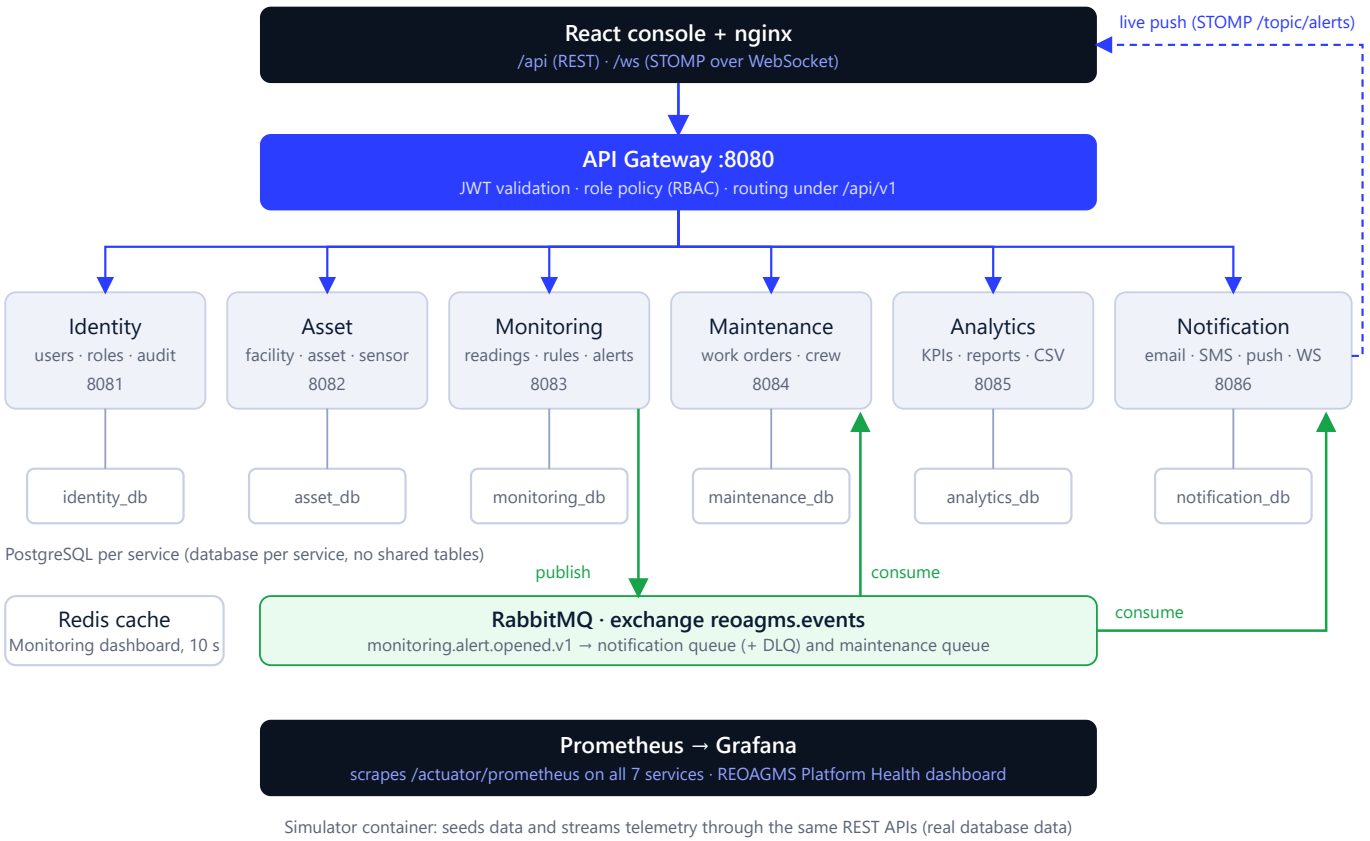
REOAGMS **at a glance**

Renewable Energy Operations, Asset and Grid Management System · EC8208 Software Architecture · Group 9 · University of Ruhuna



The flow in one line

Sensor reading → **Monitoring** checks the rules → alert stored once → event on **RabbitMQ** → **Maintenance** opens a corrective work order, and **Notification** records and pushes it to the browser in real time → **Analytics** turns the data into KPIs and reports.

Architecture style

Hybrid: layered inside each service (controller → service → repository), client–server between the console and the platform, microservices as the deployment structure, and event-driven messaging between services.

Design patterns **used here**

Verified against the code (counts are from the backend source).

Pattern	Where it appears
Singleton	Spring beans are singleton-scoped by default: one shared instance of every service, repository, controller, `JwtService`, `PushDispatcher` and config class per application context. We did not hand-write a `getInstance()`; the container guarantees it, which is thread-safe and testable.
Dependency Injection / IoC	Constructor injection everywhere; beans are wired by Spring, so classes depend on interfaces, not on how they are built.
API Gateway	Spring Cloud Gateway: single entry point, JWT validation and role policy (cross-cutting concerns offloaded from the services).
Microservices + Database per service	Six services, six PostgreSQL databases, no shared tables; cross-service references are UUIDs.
Layered architecture	Controller → Service (interface + `*ServiceImpl`, 28 of them) → Repository, in every service.
Repository	Spring Data JPA repositories (22) hide persistence behind interfaces.
Specification	JPA Specifications (11 files) build dynamic search filters for assets and work orders.
DTO	Separate request and response objects (21 dto packages) keep entities out of the API.
Builder	Lombok `@Builder` on 57 classes for readable object construction.
Publish / Subscribe (event-driven)	Monitoring publishes `monitoring.alert.opened.v1` to a RabbitMQ topic exchange; Notification and Maintenance each consume from their own queue, with a dead-letter queue.
Observer	The browser subscribes to STOMP topics `/topic/alerts` and `/topic/notifications`; the server notifies all subscribers.
Strategy	Notification channels (`EmailDispatcher`, `SmsDispatcher`, `PushDispatcher`) share a dispatcher role and are chosen by `NotificationChannel`.
Chain of Responsibility / Filter	`AuthenticationFilter` at the gateway and `JwtAuthenticationFilter` in Identity process each request in a filter chain.
Cache-aside + graceful degradation	`@Cacheable` dashboard summary in Redis (10 s); a cache error handler falls back to PostgreSQL if Redis is down.
Idempotent consumer	Maintenance stores the alert ID and skips an asset that already has an active corrective order, so redelivery creates no duplicate.
Centralised exception handling	`@RestControllerAdvice` in each service (10 classes) returns one consistent JSON error shape.
Scheduler / retry	`@Scheduled` retry of failed notifications, up to 3 attempts, 60 s apart.

If asked about Singleton directly: "In Spring every bean is a singleton by default, so each service and repository exists once per application. We rely on the IoC container instead of a manual singleton, which avoids global state and keeps classes easy to unit-test with mocks."

Likely questions **and answers**

Q. Why microservices instead of a monolith?
Independent deployment and scaling, fault isolation, clear business domains and one owner per service. The cost is more moving parts, which Docker Compose and the gateway keep manageable.

Q. Why an API Gateway?
One entry point for authentication and authorisation, simple routing, and the browser never talks to services directly. Security rules live in one place.

Q. How do services communicate, and why two styles?
REST when an answer is needed now (Analytics reading other services). RabbitMQ events when the sender should not wait or depend on receivers (alerts), which gives loose coupling and resilience.

Q. How do you handle data consistency without a shared database?
Eventual consistency: each service owns its data, and events carry changes. There is no distributed transaction; consumers are idempotent and failed messages go to a dead-letter queue.

Q. What happens if a service or RabbitMQ consumer is down?
Queues are durable, so messages wait and are processed when the consumer returns. Other services keep working, which is the point of fault isolation.

Q. What if Redis fails?
The cache error handler swallows the error and PostgreSQL answers; the API stayed under a second in our test.

Q. Why RabbitMQ and not Kafka?
Our need is reliable routing of discrete events to a few consumers, with dead-lettering and simple operation. Kafka suits very high-volume streaming and replay, which we do not need.

Q. How is security designed?
JWT (HS256) with a role claim, BCrypt password hashes, role policy at the gateway (401 without a token, 403 by role), and an audit log of logins and user changes. Self-registration always gets the lowest role.

Q. How do you scale it?
Services are stateless, so run more instances behind the gateway. RabbitMQ lets consumers scale as competing consumers, Redis offloads reads, and starter Kubernetes manifests are in `k8s/base`.

Q. How is the system observed?
Spring Boot Actuator exposes Prometheus metrics on every service; Prometheus scrapes all 7 targets and Grafana shows the provisioned Platform Health dashboard.

Q. Which quality attributes did the architecture target?
Modifiability and independent deployability (microservices), availability (isolation, durable queues, cache fallback), security (gateway RBAC), performance (cache, paging, async events) and observability.

Q. How do you version events?
The alert event carries a version in its name and a schema version field (`monitoring.alert.opened.v1`), so a v2 can run beside v1; the contract is in `docs/INTEGRATION_CONTRACTS.md`.

Q. How did you test?
About 60 unit test methods across modules, plus an end-to-end check of reading → alert → RabbitMQ → work order → notification → push, an RBAC matrix check and a Redis-outage check.

Q. What would you improve next?
More unit tests for Identity and Maintenance, OpenAPI for every service, a real SMTP and SMS provider, secrets management and TLS, and a circuit breaker on service-to-service REST calls.

Q. Is the demo data real?

Yes. The simulator uses the same REST APIs as the console, so every record is validated and stored in the owning service's PostgreSQL database.